Architectures for Hardware Acceleration

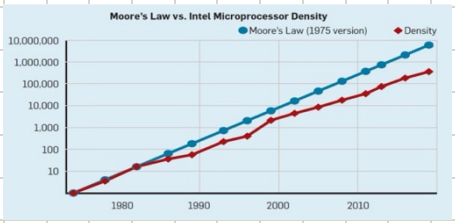
– Hitesh Pranav

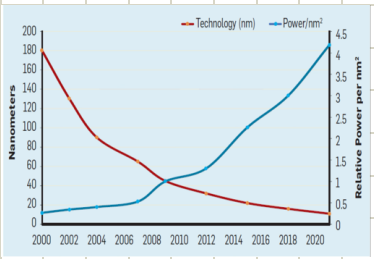
U1 Introduction to Computer Architecture Innovations

Technical Motivation for Custom Computer Architecture

→ Moore's Law

- → Moore's Law is the observation that the no. of transistors in a dense Integrated Circuit doubles every 2 years
- → This isn't a physical law, it's more of an empirical observation that became a self-fulfilling prophecy.
- → Transistor Count $\propto 2^{year/2}$ (Exponential growth in component density)
- → But in recent times, doubling time increased from 2 to 3 years & shift from smaller to smarter designs





→ Dennard's Law (MOSFET Scaling)

- → It is a scaling law which states roughly that, as transistors get smaller, power density stays constant
- → HOW! Transistors, Voltage, Current, Capacitance scale down by s

  Frequency scales by $1/s$ (So it gets faster)

  This means, Power = Voltage × Current = $s \times s = s^2$

  $$\text{Power Density} = \frac{\text{Power per transistor}}{\text{Area per transistor}} = \frac{s^2}{s^2} = \text{CONSTANT!}$$

- → Since smaller transistors gave same power density & increased clock frequency, This was the "free lunch" of computing for 30 years
- → But voltage can't be scaled down below 0.7V because of transistor threshold & leakage current becomes dominant & clock speeds can't be increased without melting chip, so 3-4 GHz became the limit.
- → Instead, we started usage of multi-core designs and dark silicon (Can't power all transistors simultaneously)

→ Amdahl's Law

- → The law describes the theoretical speedup limit when parallelizing computation
- → Speedup = $\frac{1}{(1-P)+\frac{P}{N}}$

  where P : Fraction of Program that can be parallelized

  1-P : Fraction of Program that can be sequential

  N : No. of processors/cores

- → After Dennard's law, multi-cores seemed to be a solution but it led to diminishing returns because of sequential portion 100 cores gives only around 10x speedup, not 100x

→ CISC vs RISC in post PC-Era

- → CISC (Complex Instruction Set Computer) follows the philosophy of "Make hardware do more, Simplify software"

  Basically, one instruction does LOT of work

  This makes program smaller, less instruction fetch overhead & compatibility to codes from 1978 as well

  But complex decoding logic, hard to pipeline, difficult to optimize, & power inefficient
- → RISC (Reduced Instruction Set Computer) follows the philosophy of "Keep hardware simple, Make software do more"

  Basically, one instruction does ONE simple operation

  Its simple to decode, easy to pipeline, compiler friendly, power efficient, & scalable

  But too many instructions, large code size, more no. of instruction fetches

→ This motivates us to use custom architectures because we can't rely on Moore/Dennard for automatic improvements, Amdahl limits general multi-core, RISC enables domain specific customization, 10-1000x efficiency for specific workloads

End of Moore's Law

→ Around 2000, The first signs of slowing were visible as exponential growth rate began to deviate from predictions & Manufacturing challenges at smaller nodes increased dramatically

→ Even Gordon Moore predicted in 2003 that his law would eventually fail, it was inevitable.
As of 2018, there is a 15 fold gap between Moore's Law predicted & actual capability.
This gap continues to grow as we approach fundamental limits

→ The main reason for Moore's law ending is because of physical barriers (CMOS Technology reaching fundamental limits)
The limits include Quantum Mechanics effects, Atomic Scale Barriers, Heat Density Barriers, Manufacturing Complexity & Economics, Variability etc.

End of Dennard's Law

→ Dennard's Law was the companion of Moore's Law which made it practically useful.

→ Since the computational capability of 1 mm^2 of silicon was increasing, computers became way more efficient (As mentioned before)

→ Between 1986 to 2002, exploitation of Instruction Level Parallelism was the primary architectural method for gaining performance & Improvement in speed of transistors, led to annual performance increase by 50%

→ But by 2007, Dennard scaling began to slow down significantly & effictively ended by 2012

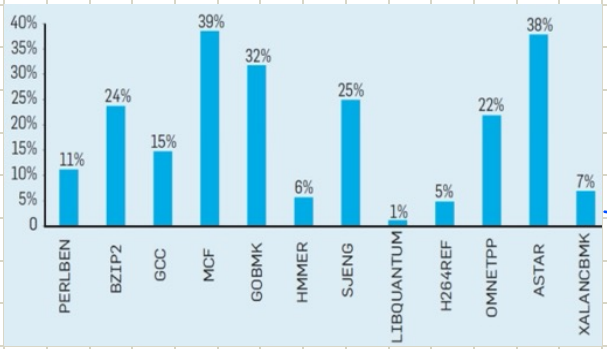
→ For very small transistor sizes, current Leakage became a serious issue, sometimes even when transistor was OFF causing chip to heat up & reliability issues.

→ ILP Dead End

→ Before multicore, architects tried making individual cores faster using ILP (Execution of multiple instructions simultaneously)

→ In a modern processor consisting of cores with 15-stage pipeline issuing 4 instructions per clock cycle, which is 60 instructions in-flight at any moment out of which 15 of those are branches.
In order to keep pipeline full, processor predicts branch outcomes & speculatively executes information. Now the real problem starts.

→ If prediction is correct, performance increases with minimal cost, improving performance
But if it is wrong, all speculated instructions are thrown away, energy is wasted & processor must restore state.



Wasted Instr. as a % of all instr. completed on Intel core i7 for a variety of SPEC integer benchmarks

To limit wasted work to 10%, processor must predict each of the 15 branches correctly 99.3% of the time
The graph shows that on an avg, 19% of inst. are wasted & actual energy wastage is even higher

→ Multicore Era

→ After ILP, architects shifted to multicore (putting multiple simpler cores on one chip)

→ But it has its own limitations:
Amdahl's law limits speedup; power scales with cores (w/o Dennard scaling, doubling cores roughly doubles power); Thermal Dissipation Power limits how much power can be removed as heat

→ This lead to Dark Silicon era where processors must slow clock rates & turn off idle cores to prevent overheating

→ CISC vs RISC Shift

→ During PC Era (CISC dominated), Intel/AMD shipped 350 million x86 processors at its peak
This made x86 win on software compatibility & economics by scale

→ Later on, during post PC Era (RISC dominated), Apple launched iPhones using ARM (RISC) processors & Mobile designers valued die area & energy efficiency as much as performance

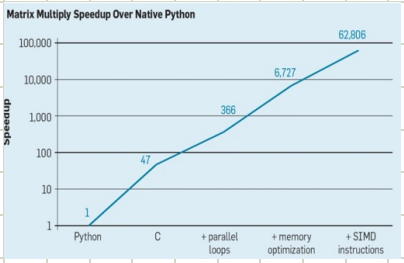
→ IoT increased the need for trade-offs in size, power & cost causing the drop in x86 shipments & RISC processors skyrocketed to 20 billion chips

Necessity for Custom Architectures

→ General purpose processors are designed to run any program reasonably well but at massive costs (as seen before)

→ Now take an example of Matrix Multiplication in Python

	Optimization Step	Speedup Factor	Cumulative Speed-up
1)	Original Python	1x	1x
2)	Rewrite in C	47x	47x
3)	Parallel loops (multicore)	7x	329x
4)	Memory layout optimization	20x	6580x
5)	SIMD hardware extensions	9x	62000x



This example illustrates the enormous gap b/w general-purpose execution & specialized hardware

For real world problems, factors of 100x - 1000x are achievable

DSAs (Domain Specific Architecture)

→ A more hardware centric approach is to design architectures tailored to a specific problem domain & offer significant performance gains

→ They are often called Accelerators as they accelerate some of an application compared to entire application in general purpose CPU

→ They also achieve better performance as they are more tailored to the needs of an application

ex: GPU (Graphics processing units), Neural Network processors for deep learning, processors for software-defined networks

The Reason for DSAs outperforming General purpose CPUs

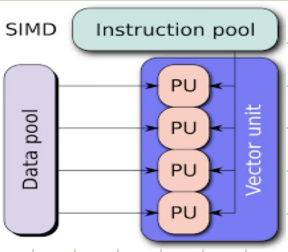
→ DSAs (Domain Specific Architectures) achieve higher performance & greater energy efficiency through 4 main advantages:

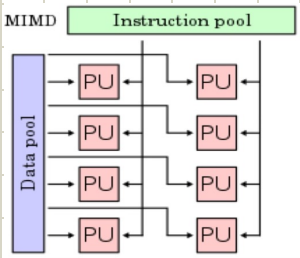
i) Efficient Form of Parallelism

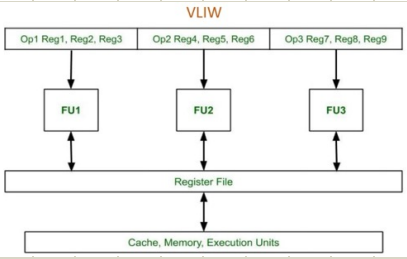
→ General purpose CPUs use MIMD (Multiple Instruction, Multiple Data) which is flexible but expensive

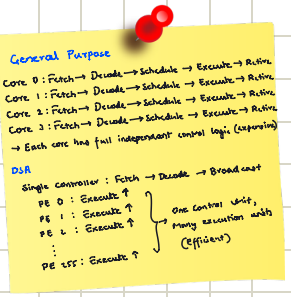
→ DSAs instead exploit SIMD (Single Instruction, Multiple Data) which is far more efficient as one instruction stream controls many processing units operating in lockstep.

→ Even though MIMD is more flexible than SIMD, DSAs use VLIW (very long instruction word), where multiple operations are packed into single wide instruction. The compiler does scheduling compile-time, eliminating need for complex runtime hardware.









ii) More efficient use of Memory Hierarchy

→ Memory access is more expensive than computation

Operation	Relative Energy Cost
32-bit integer ADD	1x
32 KB cache access	~ 200x
Last-level cache access	~ 2000x
Off-chip DRAM access	~ 20000x

→ General Purpose CPUs handle unpredictable memory patterns with hardware managed cache, consuming 50% of processor's total energy

→ DSAs use software-controlled memory hierarchies instead making it far more efficient & no wasted energy on cache tag lookups, replacement policies, coherence protocols

ex: Google TPU v1 uses 24 MB of local memory instead of caches - approximately double the on-chip memory of a comparable general-purpose CPU, and far more energy-efficient

iii) Reduced Precision when adequate

→ General-purpose CPUs always support 32-bit & 64-bit integer & floating point operations but it isn't required by many app^n's

→

	Application	Precision Needed	Wasted in GP CPU
1)	DNN Interface	4-bit, 8-bit or 16-bit integer	75-87% of datapath
2)	DNN Training	16-bit or 32-bit FP	50% of datapath
3)	Graphics	16-bit FP often sufficient	50% of datapath
4)	Signal Processing	16-bit fixed point often sufficient	50-75% of datapath

→ Using lower precision provides double benefits

1) Computational throughput - In same silicon area, 4x as many 8-bit multiples as 32-bit multiples
2) Data throughput - Smaller data means more fits in on-chip memory & more can be transferred per memory access

iv) Domain Specific Languages enable Efficient Mapping

→ DSAs need DSLs because extracting parallelism & memory access structure from general-purpose languages like python & C, is too difficult for compilers but DSLs make vectors, matrix & sparse operations explicit

ex: MATLAB

Open Architectures

→ It is an ISA thats publically available, free to implement, modifiable by designers

→ Unlike proprietary architectures, designers can build processors w/o paying licensing fees

ex: RISC-V, NVDLA

RISC-V

→ or Reduced Instruction Set Computer, 5th architecture developed at University of California, Berkley

→ One of the most important features of RISC-V is its modularity

Structure: Base ISA
+
Optional Extensions

→ Base ISA contains minimum instructions required to run a system

Variants include RV32I & RV64I

→ Option extensions include M, A, F, D, C.

This modularity allows for different features on different applications

→ Traditional architectures follow upward Binary Compatibility where future processors must run all software written for older processors. But RISC-V doesn't force backward compatability & makes new extensions optional

→ RISC-V is designed to be simple with only around 137 instructions which allows easier processor design, verification, lower development cost & easy implementation

→ RISC-V reserves a large opcode space for custom architectures (DSAs)

Nvidia Deep Learning Accelerator

→ This is another example of open architecture accelerator by NVIDIA in 2007

→ It was designed to accelerate deep learning inference & neural n/w computations

→ It is scalable, configurable & open source

→ Open architectures enable faster innovation, lower hardware development cost, more flexible processor design & support for hardware accelerators

FPGA (Field Programmable Gate Array)

→ FPGA is a semiconductor device that can be configured by user after manufacturing to implement any digital circuit.

→ Unlike ASIC (Application Specific Integrated Circuit) which is permanently fixed at the factory, FPGA can be reprogrammed

→ FPGAs offer hardware-level performance while retaining reprogrammability

→ General Structure of FPGA

→ It consists 3 main types of resources :

i) Logic elements (Brain cells)

→ Each logic element can be programmed to implement a small logic function. The basic building block is Look-up table (LUT)

ii) I/O Blocks (Interface)

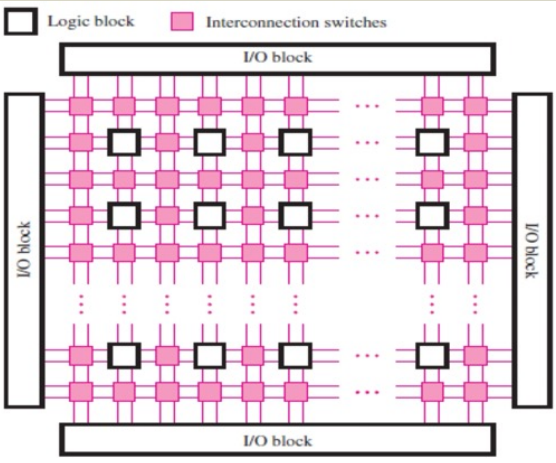
→ They connect internal FPGA fabric to physical pins of package.

→ They handle voltage levels, signal standards, & high speed interfaces

iii) Interconnection Switches (Nervous System)

→ These are organized as horizontal & vertical routing channels b/w rows & columns of logic blocks.

→ Programmable switches allow any logic element to be connected to any other logic element through routing fabric



↳ Arranged as a 2D Grid of logic blocks

→ FPGA logic elements

→ The fundamental units used to implement logic circuits are :

i) Lookup Table (LUT)

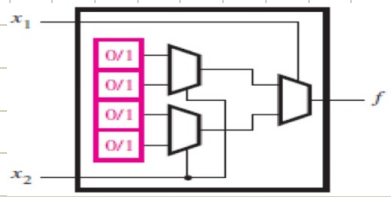
ii) Multiplexer

iii) Flip-flop (Register)

→ Allow for combinational & sequential logic

→ Look-Up Table (LUT)

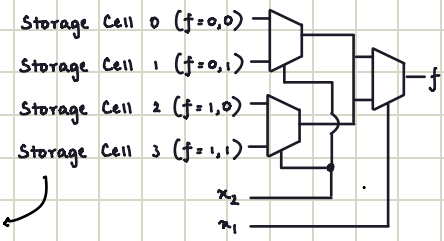
→ It is the heart of FPGA logic as it implements any logic function by storing truth table of that function in memory cells



⇒ 2-input LUT ⇒

x_1	x_2	$f(x_1, x_2)$
0	0	Cell 0 ⇒ 1
0	1	Cell 1 ⇒ 0
1	0	Cell 2 ⇒ 0
1	1	Cell 3 ⇒ 1

⇒ Multiplexer Tree



4 Storage Cells

→ LUTs behave like programmable combinational logic

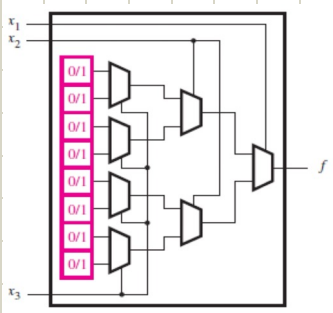
→ In general, for n-input LUT, No. of storage cells = 2^n, where n : no. of inputs

2 input LUT ⇒ 4 Storage Cells

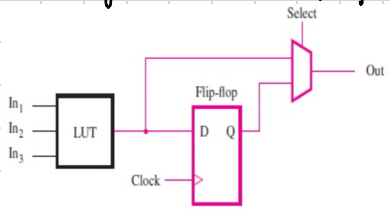
3 input LUT ⇒ 8 Storage Cells

4 input LUT ⇒ 16 Storage Cells ⟶ 4-6 i/p LUTs are used by modern FPGA

→ 3 input LUT :



→ FPGA logic element with flip flop



⟶ Flip-flops are used so that FPGA can support sequential logic

→ How to implement a circuit on FPGA?

i) Take a circuit and divide into small logic functions so each function must inside one LUT

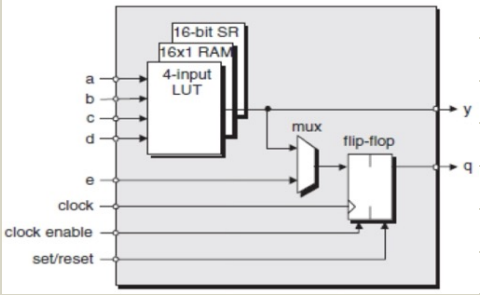
ii) Program LUTs to store truth tables

iii) Program the routing n/w to connect LUT outputs

→ Xilinx Logic Cell (LC)

→ In Xilinx FPGAs, the most basic block is called Logic cell which consists of 4input LUT, Multiplexer & Register

→ It can behave as 16x1 RAM or 16-bit shift register



→ Intel (Altera) Logic Element

→ Intel FPGAs use logic elements which are conceptually similar to logic element

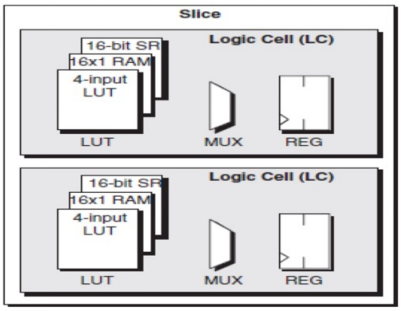
→ Differences can be seen in routing architecture, control logic, internal multiplexers but overall idea is same

→ Slice

→ Slice contains multiple logic cells

ex: 2 LC per slice / 4 LC per slice etc.,

→ They provide logic, registers, routing

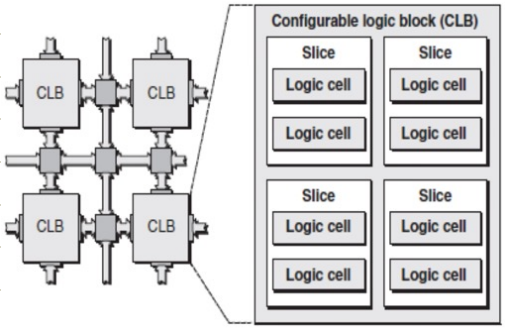


→ Configurable Logic Block

→ Higher level structure

→ In Xilinx, CLB (Configurable Logic Block)
In Intel, LAB (Logic Array Block)

→ CLB contains multiple slices, routing resources, control logic etc.,



→ Major FPGA Families (Xilinx 7 Series)

i) Spartan 7 - Low cost, low power, small footprint

ii) Artix 7 - Low power, High DSP & Logic throughput, serial communication support

iii) Kintex 7 - Good price-performance ratio, higher logic capacity

iv) Virtex 7 - Highest performance, Largest FPGA capacity

→ Modern FPGA features

i) BRAM (Block RAM)

→ Dedicated on-chip memory

→ It has configurable memory size & supports different data widths

→ It stores data using flip flops

→ FPGA Development Tools - Vivado, HLS, SDK, Logic Analyser

→ High Speed serial links ⇒ Data Rates : 600 Mbps to 28 Gbps
used for chip-to-chip communication & networking

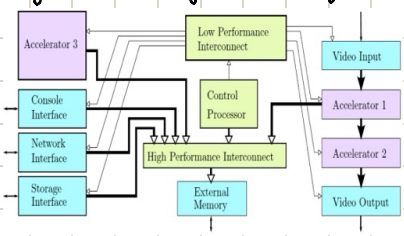
→ XADC (Analog to Digital Convertor) - dual 12 bit ADC, sampling rate = 1MSPS, temp. monitoring, voltage sensing

→ FPGA for hardware acceleration

→ They allow for parallel hardware implementation

ex: Image processing, N.Ns, Signal processing, Motor control etc.,

→ FPGA can implement PWM generation, feedback control, signal processing
It helps in deterministic timing, real-time performance, parallel comp



Hardware Accelerators

→ They are designed for Machine Intelligence Workloads as existing silicon architectures aren't efficient for them

ex: Google TPU, Intel OpenVINO, NDLA, xACC

→ Machine Intelligence (AI) Workloads have 2 fundamental capabilities

i) Approximate Learning
   → The process of working with uncertain data which require probably correct answers, not perfect ones
   ex: Image processing, Speech Recognition, Recommendation systems etc.,

ii) Learning
   → The process of extracting patterns from previous data
   → In ML,
   Training Data ⟶ Build Model ⟶ Predict Results

→ Graph Structure of AI Models
   → Many models are represented as graphs. ex: Neural N/ws consists nodes & edges
   → Graph representation exposes massive parallelism & many nodes compute simultaneously

→ Low Precision Computing
   → Many tradition computing devices use high precision (32 & 64 bit) but AI workloads require low precision (FP16, INT8, FP32)
   → This is because many small probabilities combine together & final o/p becomes accurate through aggregation
   → Opposite to HPC :3

→ With all the above requirements, specialised hardware is required as they provide much higher performance & lower energy consumption compared to general CPUs (10x to 1000x)

→ Future trends expect many custom accelerators (mobile, server processors etc.,). They are called Heterogeneous Architectures

→ Parallelism - This has improved computer architecture a lot

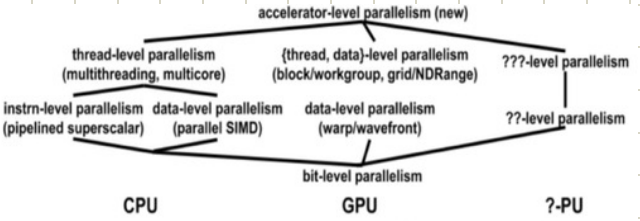
Bit Level Parallelism
→ Operate on multiple bits
ex: 64 bit processor - 64 bits processed in one operation
→ Now most processors use 64 bit data paths

Instruction Level Parallelism
→ Allows processor to execute multiple operations simultaneously
→ Techniques include pipelining, superscalar execution etc.,
→ It is hidden from programmers

Thread Level Parallelism
→ It uses multiple processor cores
→ Sets the base of multicores
ex: Intel Xeon (upto 128 cores)

Data Level Parallelism
→ Perform same operation on many data items
→ Used in GPUs & SIMD
ex: SIMT (Single Instr, Multi. Thread)

→ Parallel Lattice
   → Modern processors combine multiple types of parallelism



→ Architecture Level Parallelism
   → Because Moore's Law & Dennard's Scaling are slowing, a new form of parallelism is emerging
   → ALP refers to multiple hardware accelerators running concurrently
   ex: CPU GPU AI Accelerator Video Decoder Image Processor DSP
      ↳ Each unit accelerates a specific task
   → Smartphone use ALP extensively

→ Challenges in Hardware Accelerator Design

→ Despite advantages, they have multiple disadvantages

	CPUs (AP)	Display	Media Scaler	GPU	Image Signal Proc.	JPEG	Pixel Visual Core	Video Decoder	Video Encoder	Dozens More
Photo Enhancing (HDR+)	X	X		X	X	X	X			
Video Capture	X	X		X	X				X	
Video Capture (HDR)	X	X		X	X				X	
Video Playback	X	X	X	X				X		
Image Recognition	X	X	X	X						

→ C1 : Accelerator Specialization

→ Different Applications need different accelerators

→ But hardwore resources are limited :
chip area, power, cost

→ Thus architects must choose which accelerators to include

→ The current approach process is often manual

Steps :

i) Identify performance bottleneck
ii) Design hardware accelerator
iii) Integrate onto chip

→ This approach is slow & expensive

→ Future solutions require automatic accelerator design tools

→ C2 : Accelerator Concurrency

→ Many accelerators run simultaneously

Ex : In smartphone ⇒ Camera capture, Image processing, AI detection, Video Enhancing ⇒ All run at same time

→ The problem for this is that Accelerator competes for shared resources like DRAM BW, Network on Chip, Cache etc.,
This leads to resource contention

→ Network-on-Chip (NOC) is used to communicate b/w components in most modern chips
Its like LAN n/w inside processor to connect CPU, GPU, Memory, Accelerators

→ C3 : Accelerator Communication

→ Accelerators must communicate w each other

→ The typical flow :

CPU (Accelerator Setup) ⟶ Accelerator executes ⟶ Accelerator sends interrupt ⟶ CPU moves data ⟶ Next accelerator starts

This makes CPU as master controller

→ But the issue is data movement consumes lots of energy (Upto 62.7% System Energy spent moving data from memory & computing units)

→ This reduces efficiency

→ Architects must explore shared memory architectures, hardware coherence protocols, direct accelerator communication etc.,
to minimize unnecessary movement

→ C4 : Accelerator Programmability

→ Programming accelerators is difficult & each accelerator often requires special SDK & programming framework

ex : GPU ⟶ OpenCL/CUDA ; DSP ⟶ FastRPC ; CPU ⟶ C/C++

→ This reduces portability across devices

→ Programming Complexity

Simple Kernel : 30 lines of code for setup
Accelerator : 100+ lines of code for setup ⟶ Difficult

→ To exploit ALP, we need better tools
Possible solutions are unified programming framework, portable libraries, domain-specific language etc.,

ex: Open MP (↓ 1st programming), MPI (↓ Distributed Sys.), MapReduce, Halide (↓ Cluster computing) ⟶ Image Processing DSL

So what must we do ?

→ Adapt quickly to ALP as future relies on it

→ Qualcomm - mobile AI
ARM - IoT Processors
Mobile Eye - Autonomous Vehicle
Google - ML
Microsoft - Datacenter AI

High Level Synthesis

→ It is a design methodology used in hardware design & hardware acceleration to convert high level algorithm descriptions into hardware implementation

→ It allows engineers to use SystemC instead of manually writing RTL code

→ Datapath vs Control Path

→ Digital systems are divided into datapath & control path to implement algorithms in hardware

Datapath

→ Performs data processing / transformations
ex: ADD, SUB, MUL, DIV

→ These operations manipulate data values

→ So datapath contains mainly arithmetic units, registers, multiplexers, ALUs

Control path

→ Manages data movement & decision making
ex: MOV, LD, ST, PUSH

→ These instructions control when & where data moves

→ Control path contains state machines, control logic and decision results

→ Algorithms : Datapath vs Control Path Dominance

↳ Categorized based on which component dominates

Algo. Type	Dominant Comp.
Signal Processing	Datapath
Networking Protocols	Control Path
Queue Management	Control Path
Modem Processing	Datapath

→ Importance of HLS

→ HLS works for Datapath & Control Path heavy algorithms

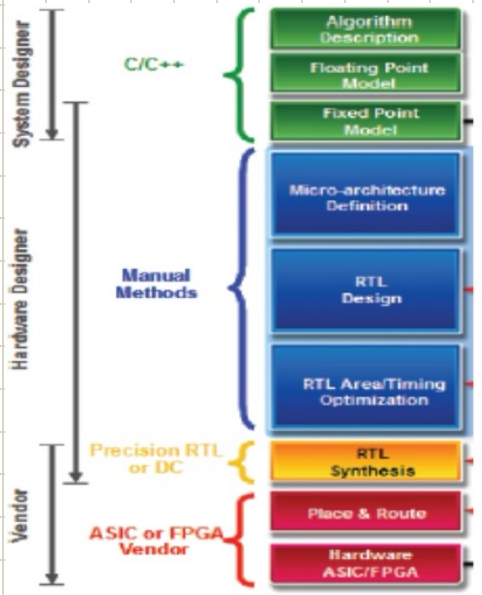
→ Widely used in communication systems, multimedia processing, security systems etc.,

→ So basically HLS converts high-level algorithm description into RTL hardware design automatically

→ Designers write algorithms in C/C++/SystemC instead of RTL to generate registers, datapath, control logic, interfaces etc.,

→ Tasks performed by HLS

i) Concurrency Detection
ii) Register Insertion
iii) Control Logic Generation
iv) Interface Generation
v) Memory Mapping
vi) Resource Mapping



→ Modern HLS Tools – Vivado HLS, Stratus HLS, Catapult HLS
– Support C/C++ input, SystemC modelling, automatic RTL Generation

→ Advantages of HLS

i) Higher Productivity
ii) Faster Design
iii) Easier Design exploration
iv) Better optimization

→ Limitations

i) Generated hardware may not be optimal
ii) Designer must understand both hardware & software concepts
iii) Some algorithms are difficult to synthesize

U2 Basic Building Blocks of Accelerator Logic

Sorting

→ It is a fundamental operation in many computing systems
It arranges data elements in ascending / descending order based on comparision

→ There are many applications (Databases, Data mining, N/w packet processing, Pattern Recognition etc.,) which have very high throughput & software sorting is slow

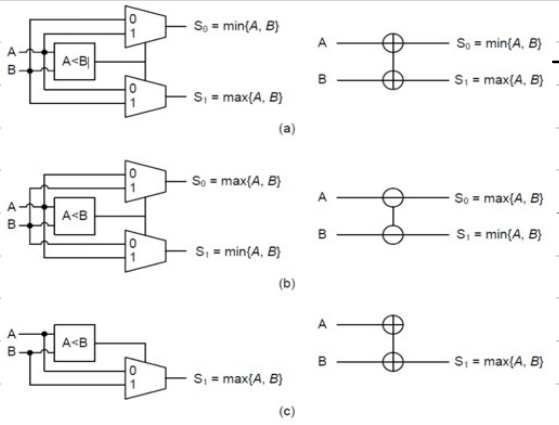
→ For extremely high speed applications, sorting is implemented using ASICs & FPGAs which allow massive parallel comparisions
ex: High Energy Physics ⟶ Sort 128 - 256 samples in 100 ns

→ **Modular (Parallel) Sorting Networks**

→ Sorting N/w is a fixed n/w of compare - and - exchange blocks that sorts i/p in parallel

→ The structure of n/w is fixed & doesn't depend on input values
Thus hardware always performs same sequence of comparisions

→ **Compare - And - Exchange** block (CAE) compares 2 values & outputs them in sorted order



⟶ Increasing CAE block
Input : 7,3
Output : 3,7
↘ Both are identical, only wiring diff.
↗
⟶ Decreasing CAE block
Input : 3,7
Output : 7,3

⟶ Max block
Input : 3,7
Output : 7

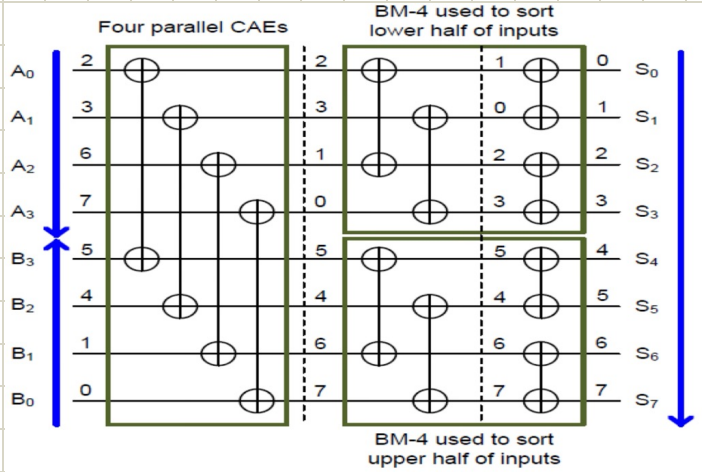
↳ Each CAE block consists of comparator & 2 multiplexers

→ **Sorting Network Structure**

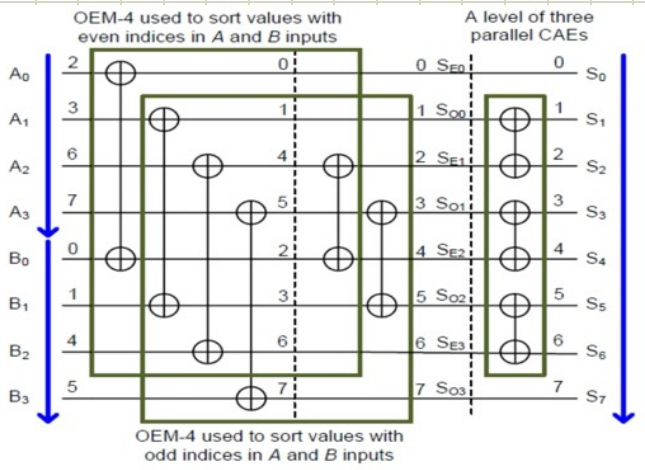
→ It consists of multiple stages (Stage consists of multiple CAE blocks working in parallel)

→ It's latency depends on n/w depth (Latency ∝ No. of CAE Stages)

→ Some popular Sorting Networks include Bitonic Sorting Network & Odd-Even Merge Sorting Network



↳ Bitonic Sequence - Sequence that first increases & then decreases
ex: 2, 3, 6, 7, 5, 4, 1, 0
↳ Increasing : 2, 3, 6, 7
Decreasing : 5, 4, 1, 0
⟶ Bitonic Merge Unit combines the bitonic sequence into fully converted seq.
↳ Output : 0, 1, 2, 3, 4, 5, 6, 7



↳ Odd-Even Merge Sorting N/w : 1) divides data into odd-index elements
2) divides data into even-index elements
3) Sort seperately
4) Merge odd & even comparisions

→ Verilog Model for Sorting

```
`timescale 1ns/1ps
module comp_swap_asc (
    input clk,
    input [`DATA_WIDTH-1:0] a,
    input [`DATA_WIDTH-1:0] b,
    output logic [`DATA_WIDTH-1:0] a_o,
    output logic [`DATA_WIDTH-1:0] b_o

);
    logic  [`DATA_WIDTH-1:0]  a_next;
    logic [`DATA_WIDTH-1:0] b_next;

    always_comb begin

        if(b<a) begin
            a_next = b;
            b_next = a;
        end
        else begin
            a_next = a;
            b_next = b;
       end
    end

    always_ff @(posedge clk) begin

        a_o <= a_next;
        b_o <= b_next;

    end

endmodule
```

```
module comp_swap_dsc (
    input clk,
    input [`DATA_WIDTH-1:0] a,
    input [`DATA_WIDTH-1:0] b,

    output logic  [`DATA_WIDTH-1:0] a_o,
    output logic [`DATA_WIDTH-1:0] b_o
);


    logic [`DATA_WIDTH-1:0] a_next;
    logic [`DATA_WIDTH-1:0] b_next;

    always_comb begin

         if(a<b) begin
             a_next = b;
             b_next = a;
         end
         else begin
             a_next = a;
             b_next = b;
         end


    end

    always_ff @(posedge clk) begin

         a_o <= a_next;
         b_o <= b_next;

    end


endmodule
```

```
`timescale 1ns/1ps
module test_sort;

logic [`SORT_SIZE-1:0][`DATA_WIDTH-1:0] data_in;
logic [`SORT_SIZE-1:0][`DATA_WIDTH-1:0] data_out;
logic clk;

always begin
         #10
         clk =  ~clk;
end


    bitonic_sorter bs(clk,data_in,data_out);
initial begin

    clk = 0;
    $display("testing with input size of %d", `SORT_SIZE);

    for(int i=0;i<`SORT_SIZE;i++) begin
        data_in[i] = {$random,$random};
        $display(data_in[i]);

    end

    #500000000
    $display("------------------------------");
    for(int i=0;i<`SORT_SIZE;i++) begin
       $display(data_out[i]);
    end

    $finish;


end
endmodule
```

Systolic Architectures

→ It is a hardware design consisting of many simple processing elements (cells) that are connected in a regular structure & Data flows rhythmically b/w those cells & each cell performs simple computation

→ Modern applications require very high computation performance which systolic architecture provides:

i) Map high level algorithms to hardware

ii) Perform parallel computations

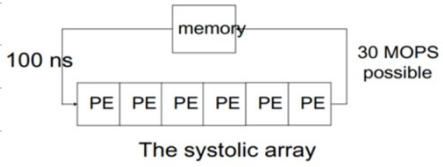
iii) Reuse data efficiently

→ Key Architectural issues in special purpose systems

i) Simple & Regular Design

ii) Concurrency & Communication

iii) Balancing computation with I/O



→ PE: Processing Element

→ Basic Principle

→ Systolic system consists of multiple processing cells, interconnected in a regular grid & data flows in cells synchronously

→ Cells perform simple operation, computations occurs as data flows

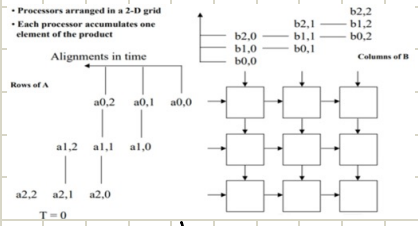
→ Systolic Array

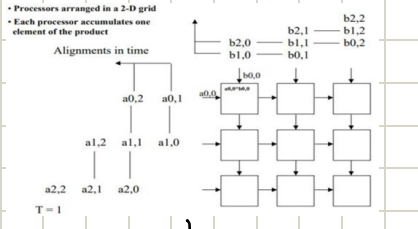
→ It is a n/w of processing elements arranged in regular structure (1D/2D grid)

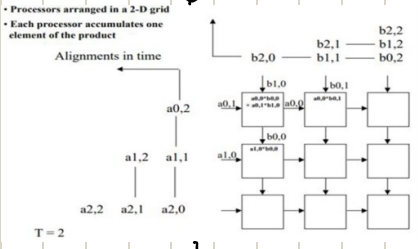
ex: Matrix Multiplication

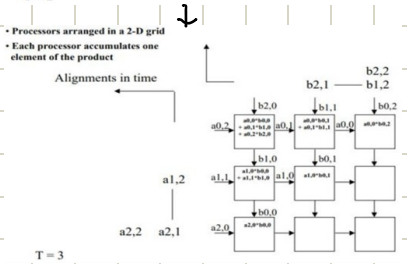
$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \times \begin{bmatrix} j & k & l \\ m & n & o \\ p & q & r \end{bmatrix} =$$

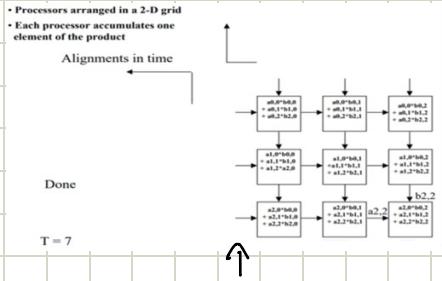
$$\begin{bmatrix} aj + bm + cp & ak + bn + cq & al + bo + cr \\ dj + em + fp & dk + en + fq & dl + eo + fr \\ gj + hm + ip & gk + hn + iq & gl + ho + ir \end{bmatrix}$$

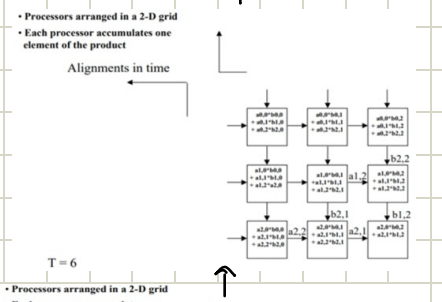


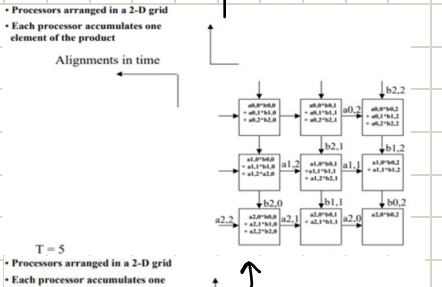


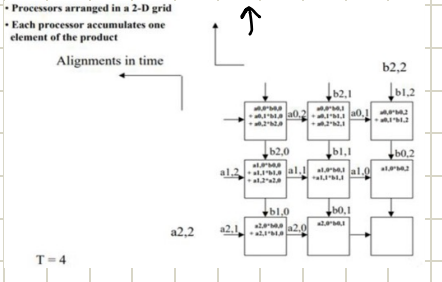












→ **Applications**

1) Matrix Multiplication
2) DSP (FIR, IIR etc.,)
3) Non-numeric Applications (stacks, Queues etc.,)

→ **Convolution**

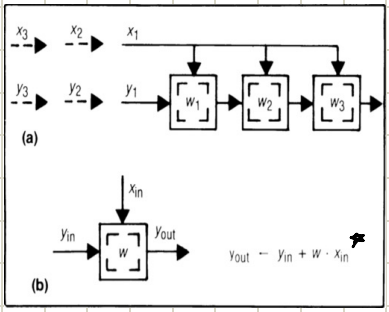
→ Assume input ⇒ $\{x_1, x_2, x_3 \dots x_K\}$
weights ⇒ $\{w_1, w_2, w_3 \dots w_K\}$
Compute output ⇒ $\{y_1, y_2, \dots, y_{n+1-K}\}$
So, $y_i = w_1 x_i + w_2 x_{i+1} + \dots + w_K w_{i+K-1}$

→ Each i/p multiplies with weights which requires high B.W

→ Instead of repeatedly fetching i/p from memory, i/p is reused across multiple cells
This reduces memory access & increases throughput

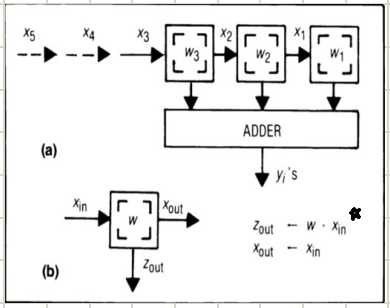
→ 3 Major Designs

i)



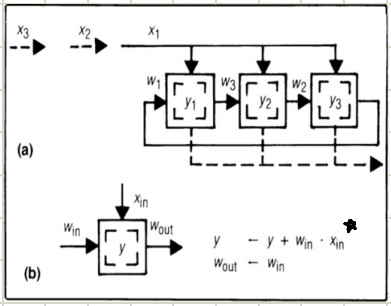
B1 - Broadcast i/ps, Move results, Weights stay

ii)



ii) B2 - Broadcast i/ps, Move Weights, results stay

iii)



iii) F - Move i/ps, Weights stay, Broadcast results

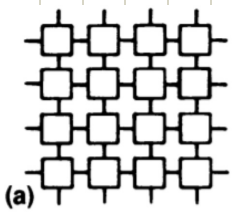
Note: Whichever moves, it moves systolically
ⓐ is systolic array
ⓑ is cell

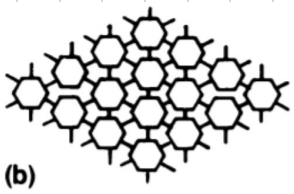
→ **Configuration Issues**

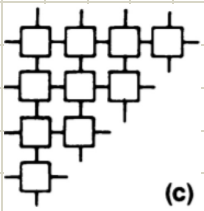
i) Processing v/s I/O Bandwidth
ii) General Purpose vs Special Purpose
iii) Cost of implementation
iv) Modularity & Expansion
v) Trade off b/w simplicity & complexity of cell

→ **2D Systolic Arrays**

i) Type R - Rectangular ii) Type H - Hexagonal iii) Type T - Triangular







→ Advantages of Systolic Architecture

i) High Throughput
ii) Massive Parallelism
iii) Simple Cell Design
iv) Predictable Data Flow

→ Why Systolic?

→ In human heart, systole = contraction phase where blood is pumped rhythmically through body
→ Similarly, Data Flows rhythmically through cells & performs operations in each cycle

→ Types of Systolic Arrays

i) Special Purpose SA - Hardwired for specific application & very high performance
ii) General Purpose SA - More flexible, Programmable operations
iii) Programmable SA - Cells can execute instr & support SIMD, MIMD
iv) Reconfigurable SA - Implemented using FPGAs

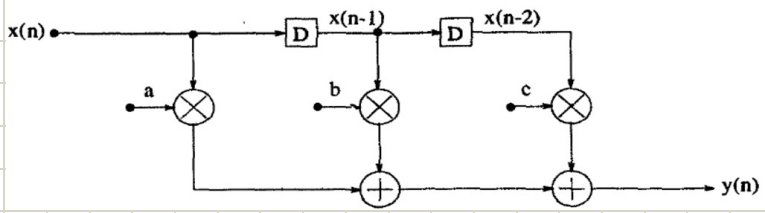
→ Implementation Issues

i) Algorithm Mapping – Designers must carefully map algorithms to arrays
– Poor mapping = Inefficiency & low performance
ii) System Integration – Systolic arrays act as coprocessors
– CPU ⟶ Memory subsystem ⟶ Systolic Array accelerator
– Extra memory buffers are needed
iii) Cell Granularity – Cell complexity varies
i) Bit level – Logical op.
ii) Word level – add/multiply
iii) Processor level – full program execution
iv) Clock Synchronization – Large arrays suffer from clock skew
– Solutions include wavefront arrays & asynchronous communication
v) Reliability – Large chips require fault tolerance & diagnostic systems

FIR Filter

→ Finite Impulse Response Filter computes weighted sum of present & past inputs

ex: 3 tap FIR ⇒ $y(n) = ax(n) + bx(n-1) + cx(n-2)$



Clock period = Sample Period
30 frames/sec
In KHz range

→ Critical Path = Longest Delay in Circuit

$T_{critical} = T_m + 2T_n$
(T_m: Multiplier Delay, T_n: Adder Delay)

→ Drawbacks

i) Changing coefficients is difficult
ii) Changing no. of taps requires complete resynthesis
iii) Hardware Cost
iv) Clock = Sample Rate (Each i/p sample requires full computation in 1 clock cycle)

→ Serial FIR filter

→ To reduce cost, architecture is serialised by making only 1 multiplier be used repeatedly instead of many

→ For K-tap filter

i) Store i/p in registers

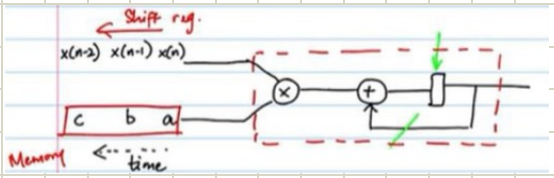
ii) One multiplier processes ⇒ $ax(n)$, $bx(n-1)$, $cx(n-2)$...

iii) Each product added sequentially

→ Hardware : 1 Multiplier, 1 Adder, Multiple Registers

→ Reduced Hardware Cost

But slow & multiple cycles req. for o/p



→ Serial FIR filter

→ Serial architecture needs control unit

→ Control path manages which i/p to use, which co-eff to use, timing of multiplication, accumulation of results

→ Components : Control unit, Multiplier, Multiplexer, Adder, Register

→ Steps : i) ctr = 0 & next = 0

ii) While True :

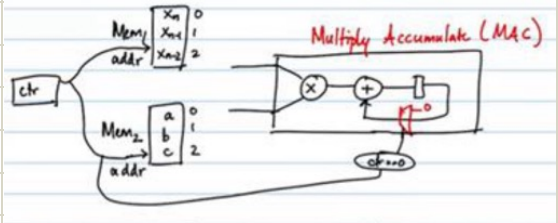
if ctr == 0 , acc = 0

mem [next] ← input

next = (next + 1) % N

mult add mem_1[ctr] , mem_2[ctr]

ctr = (ctr + 1) % N



→ Data Flow Graphs

→ Represents computation graphically

→ Nodes represent operations

S : Source

A : Adder

M : Multiplier

D : Delay

→ Original Flow : S ⟶ A ⟶ M ⟶ A

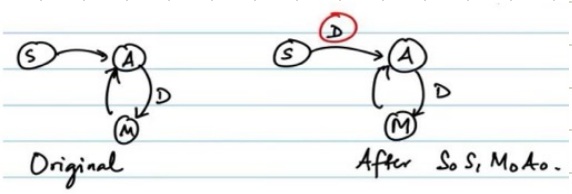
with feedback through D

→ After Scheduling : S0 ⟶ A0 ⟶ M0 ⟶ A1

Delay shifts operations to next iteration

This creates dependancy M0 ⟶ A1 ✓

M0 ⟶ A0 ✗



→ Invalid sequences : A_0 ⇒ A needs i/p from S0 & M-1 which aren't available yet

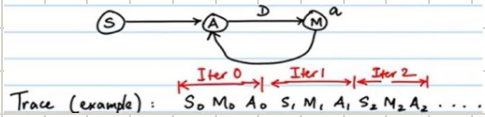
S0 M0 A0 A1 ⇒ M0 Scheduled before A0

S0 S1 M0 M1 ⇒ No dependancy on adders

→ Iteration Period

→ The minimum time b/w start of 2 consecutive iterations of an algorithm

→ Take for ex,



each step = 1 unit

S → M → A ⇒ 1+1+1 = 3 units

→ Iteration Period = Min time b/w starting 2 iterations

→ Avg Iteration Period = $\frac{\text{Time for N iterations}}{N}$

→ Parallel Architecture

→ To increase speed we split into even & odd sample streams

$x(0), x(2), x(4) \ldots$ ↳ $x(1), x(3), x(5) \ldots$

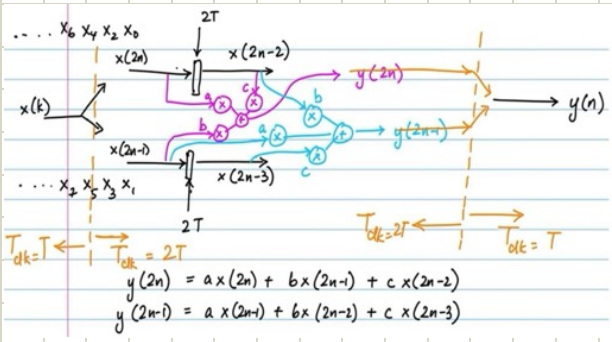
This allows computation upto $y(2n)$ & $y(2n+1)$

→ Input is split: upper → $x(2n)$, lower → $x(2n-1)$

For even o/p, $y(2n) = ax(2n) + bx(2n-1) + cx(2n-2)$

For odd o/p, $y(2n-1) = ax(2n-1) + bx(2n-2) + cx(2n-3)$

Merge $y(2n)$ & $y(2n-1)$ to get $y(n)$



Note: Make sure to set delay by 2T because system processes 2 samples per cycle

→ Clock rate: f to $f/2$

Throughput: Remains same

Power Consumption ⇒ $P = \alpha C V^2 f$

$f \rightarrow f/2$

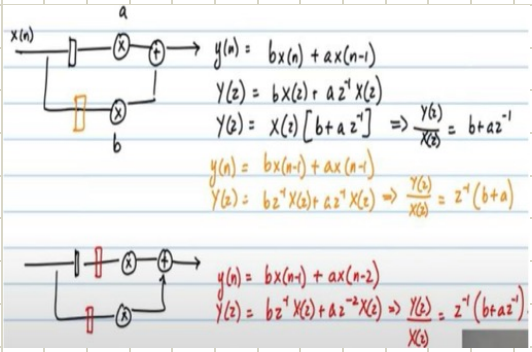
$C \rightarrow 2C$ (load capacitance)

→ Pipelining FIR

→ To increase speed, instead of x → + → + use x → reg. → + → reg. → +

So CP goes from $T_m + 2T_a$ to $\max(T_m, 2T_a)$

Clock Period ↓, Throughput ↑



→ Parallel FIR

→ Take $y(n) = ax(n) + bx(n-1) + cx(n-2)$

To compute 3 o/ps at once, substitute $n = 3k, 3k+1, 3k+2$

$y(3k) = ax(3k) + bx(3k-1) + cx(3k-2)$

$y(3k+1) = ax(3k+1) + bx(3k) + cx(3k-1)$

$y(3k+2) = ax(3k+2) + bx(3k+1) + cx(3k)$

i/p: $x(0), x(1), x(2) \ldots$

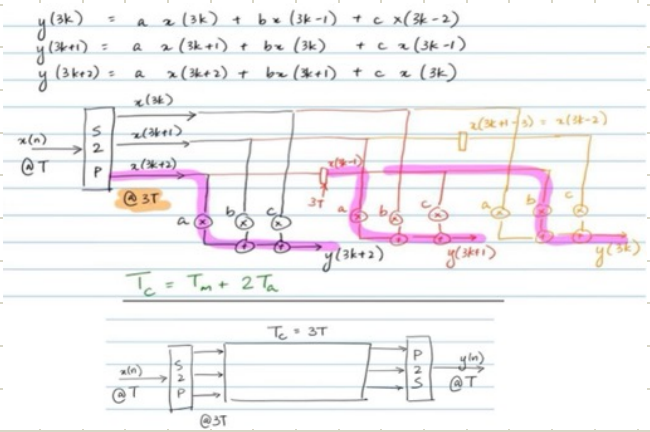
split into: $x(0), x(3), x(6) \ldots$

$x(1), x(4), x(7) \ldots$

$x(2), x(5), x(8) \ldots$

→ Time scaled to sample period = 3T

making clock run 3x slower



→ Speed ↓

Hardware ↑

Power ↓

Throughput ↑

CORDIC Algorithm

→ COordinate Rotation DIgital Computer

→ Its an algorithm to compute mathematical f^n's using shifts, additions, subtractions & LUTs

It avoids multipliers & dividers

→ To compute sin & cos,

use taylor series : $\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$

or storing all values in LUTs

but both are inefficient!

So CORDIC uses vector rotations using shift & add operations

Applications

Linear transformation:
- DFT, Chirp-Z transform, DHT, and FFT.

Digital filters:
- Orthogonal digital filters, and adaptive lattice filters.

Matrix based digital signal processing algorithms:
- QR factorization, with applications to Kalman filtering
- Linear system solvers, such as Toeplitz and covariance system solvers,.....,*etc.*

→ Main idea : Rotate vector step-by-step, instead of 1 big rotation, use multiple small rotations

Vector rotation formula : $x' = x\cos\theta - y\sin\theta$, $y' = x\sin\theta + y\cos\theta$

$$\rightarrow \begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \cos\theta \begin{pmatrix} 1 & -\tan\theta \\ \tan\theta & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

Iterative Rotation :

1) Start with (x, y) coordinates & Target angle of rotation $= 70^\circ$ (example)
2) Rotate by 45°
3) Rotate by $22.5^\circ = 67.5^\circ$
4) Rotate by $11.25^\circ = 78.75^\circ$
5) Rotate by $-5.625^\circ \approx 73^\circ$
6) Rotate by $-2.862^\circ \approx 70^\circ$ → Iterate till close enough

and store $\theta = 45, 22.5, 11.25, -5.625, -2.862$ in LUT → INEFFICIENT

→ CORDIC uses angles where $\tan(\theta_i) = 2^{-i}$ which allows multiplication by shift operations

because $x \cdot 2^{-i}$ = right shift

algorithm becomes shift + add

→ Rotate by $(\theta = 45^\circ) \Rightarrow \cos\theta_1 \begin{bmatrix} 1 & -2^0 \\ 2^0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$

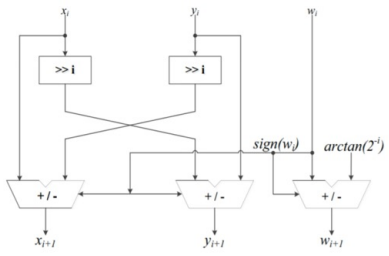
Rotate by $(\theta = 22.5) \Rightarrow \cos\theta \begin{bmatrix} 1 & -2^{-1} \\ 2^{-1} & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} x_2 \\ y_2 \end{bmatrix}$

$$\begin{bmatrix} x_n \\ y_n \end{bmatrix} = \cos\theta_1 \cdot \cos\theta_2 \dots \cos\theta_n \begin{bmatrix} 1 & -\tan\theta_1 \\ \tan\theta_1 & 1 \end{bmatrix} \begin{bmatrix} 1 & -2^{-1} \\ 2^{-1} & 1 \end{bmatrix} \begin{bmatrix} 1 & -2^{-2} \\ 2^{-2} & 1 \end{bmatrix} \dots \begin{bmatrix} x_{n-1} \\ y_{n-1} \end{bmatrix}$$

→ CORDIC Iteration Equations

$$x_{i+1} = x_i - y_i d_i 2^{-i}$$
$$y_{i+1} = y_i + x_i d_i 2^{-i}$$
$$z_{i+1} = z_i - d_i \tan^{-1}(2^{-(i)})$$

where $d_i = \begin{cases} -1, & z_i < 0 \\ 1, & \text{otherwise} \end{cases}$



→ 2 Modes of CORDIC

i) Rotation Mode

→ Rotate vector by known angle

→ I/p : $\begin{pmatrix} x_0 \\ y_0 \end{pmatrix}$ O/p : $\begin{pmatrix} x_1 \\ y_1 \end{pmatrix}$

→ Procedure :
1) Load desired angle in accumulator
2) Perform iterative rotations
3) Each iteration produces residual angle
4) Final vector rotated by θ

ii) Vectoring Mode

→ Find angle & magnitude of a vector

→ I/p : $\begin{pmatrix} x_0 \\ y_0 \end{pmatrix}$ O/p : magnitude, angle

→ Procedure :
1) Rotate vector until $y \to 0$
2) Vector lies on x-axis
3) Accumulated rotations gives angle

Hardware Security Primitives

→ Basic building blocks used to secure ICs against attacks

ex: PUF (Physical unclonable Function), TRNG (True Random Number Generator), DfAC (Design for Anti-Counterfeit)

→ These primitives help protect devices during IC manufacture, device operation, external communication etc.,

→ They prevent cloning, counterfeiting & unauthorized access

→ **PUFs**

→ Hardware circuit that produces unique digital fingerprint for each chip

→ These fingerprints come from unavoidable manufacturing variations

→ Even 2 identical chips that are identical, can never be same because of microscopic differences (transistor size, doping, gate oxide thickness, interconnect properties etc.,)

→ **Digital Footprint** - PUF behaves like biometric fingerprint for hardware

ex: Input Challenge ⟶ PUF ⟶ Output Response

→ It enables device authentication

→ **PUF applications**

i) Cryptographic Key Generation

ii) Device Authentication

iii) Anti-cloning protection

→ **Why CMOS variations are better?**

→ CMOS devices dominate because of low power, high density, low manufacturing cost

→ However, process variations are unavoidable giving PUFs an advantage

Table 12.1 Design and technology characteristics vs. security trade-off

Application/primitive	Process variation	Temperature	Power supply noise	Aging (BTI/HCI)	Wear-out (EM)
Logic/memory design	Ugly	Bad	Bad	Bad	Bad
PUF	Good	Bad	Bad	Bad	Bad
TRNG	Good	Bad	Good	Bad	Bad
Recycled IC detection	Ugly	Bad	Bad	Good	Good

→ **Process Variations**

→ During chip fabrication, several variations occur

Front-End of Line (FEOL)

→ These affect transistor transformation

ex: line-edge roughness, doping variations, oxide thickness variations, gate material granularity etc.,

→ These change transistor parameters like threshold voltage, drive current

Back-End of Line (BEOL)

→ These affect interconnections

ex: metal wire variation, dielectric variation etc.,

→ These change signal delay, R, C etc.,

→ **Environment Variations**

→ Device behavior also changes during operations

ex: temperature change, power supply noise, aging etc.,

→ Effects include change in carrier mobility, threshold voltage, drain current etc.,

Unlike manufacturing effects, these variations are temporary

→ PUF Classification

→ There are 2 types

i) Weak PUF / Physically Obfuscated Key

→ Small no. of challenge-response pairs

→ Limited CRP space

→ Usually used for key generation

ex: SRAM PUF

ii) Strong PUF

→ Many no. of challenge-response pairs

→ Large CRP space

→ Usually used for Authentication protocols

ex: Arbiter PUF, Ring Oscillator PUF

→ Controlled PUF

→ It adds extra protections

→ PUF + External Logic + Algorithm + API

→ It prevents direct access to PUF, protects against spoofing & modelling attacks

100 Chips manufactured
0110100.....1011 (32-bit) (input/challenge) = 01010110...11 (64-bit) (output/response) (chip -1)
0110100.....1011 (32-bit) (input) = 10111100...01 (64-bit) (chip -2)
0110100.....1011 (32-bit) (input) = 01111110...10 (64-bit) (chip -3)
Similarly, for all 100 chips, the output of the PUF circuit is different for same input.

→ PUF Quality Properties

1) Uniqueness

→ Different chips must produce different response

→ It is measured using inter-PUF Hamming distance

$$HD_{inter} = \frac{2}{n(n-1)} \sum_{i=1}^{n-1} \sum_{j=1}^{n} \frac{HD(R_i, R_j)}{k} \times 100\%$$

ex: 0100 & 1001

HD = 3 ⟶ Higher HD = More unique

2) Uniformity (Randomness)

→ PUF o/p bits should be evenly distributed

ex: 01010101 ⇒ Good

0000100 ⇒ Bad

Ideal: 50% 1, 50% 0

3) Reliability (Reproducability)

→ Same chip must produce same response every time

even under temperature change, power supply noise, aging etc.,

→ It is measured using intra-PUF Hamming distance

Low intra-HD = High Reliability

$$HD_{intra} = \frac{1}{m} \sum_{y=1}^{m} \frac{HD(R_i, R'_{i,y})}{k} \times 100\%$$

→ Avalanche Effect

→ Good PUF should have high diffusion property

→ Small i/p change ⟶ Large response change

This prevents attackers from being able to predict o/p

→ Randomness Testing

→ It is measured using statistical tools

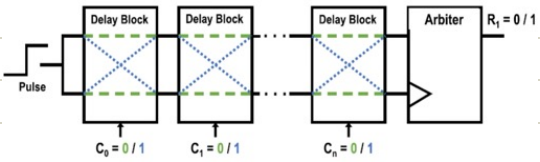
→ Common test suites: NIST test suite, DieHARD tests

These check for bias, correlation, randomness etc,

→ **Arbiter PUF**

→ It exploits random delay differences in circuit paths caused by manufacturing variations

→ Even if 2 paths are identically designed,
Path 1 delay ≠ Path 2 delay



→ **Main Architecture :**

Input Pulse ⟶ Delay stages ⟶ 2 Signal Path Race
Output Bit (0/1) ⟵ Arbiter decides which arrived first

→ **Operation**

1) Input pulse : Signal pulse injected in CKT.
2) Path selection : Each stage contains switches controlled by challenge bits (ci)
The challenge determines how signal travels through delay n/w
ex : C1 C2 C3 → Determine switching pattern
3) Signal Race : 2 signals propogate simultaneously through 2 paths
Due to manufacturing defects, Path 1 delay ≠ Path 2 delay
4) Arbiter Decision : Arbiter assigns o/p = 1 for first arrived signal
o/p = 0 for next signal

→ **Advantages :**

→ Generates response in 1 C.C
→ Large CRP space
→ Simple CMOS implementation

→ **Disadvantages :**

→ Arbiter bias
→ Layout symmetry required
→ Vulnerable to modelling attacks

→ **Ring Oscillator PUF (RO PUF)**

→ Instead of path delay, RO PUF uses freq. difference b/w oscillators

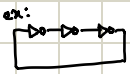
→ Frequencies aren't same due to manufacturing variations

→ **Main Architecture :**

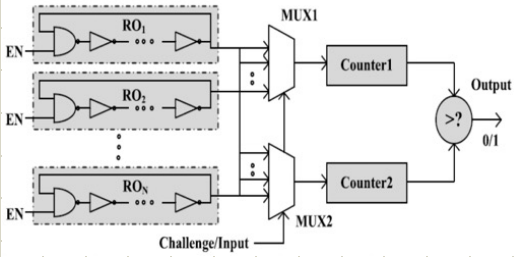
Multiple ROs, MUX, counter, comparator

→ The RO is made of odd no. of inverters connected in loop

ex:



This produced continuous oscillation



→ **Operation**

1) Challenge selects 2 ROs
2) Each RO runs for fixed time & no. of oscillations are counted
3) Compare ⇒ faster : o/p = 1 ; slower : o/p = 0

→ Rule : If you need 1 response bit, minimum 4 challenge i/ps
8 response bit, minimum 32 challenge i/ps

→ **Advantages :**

→ Easy to implement on FPGA
→ No strict rounding symmetry req.

→ **Disadvantages :**

→ Large Area
→ Slow response generation
→ High power consumption

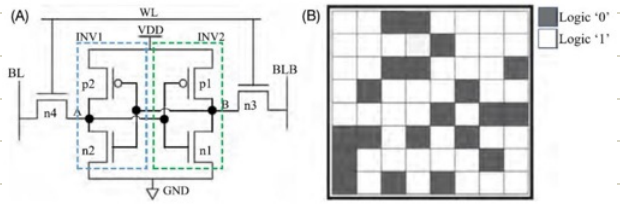
→ SRAM PUF

→ Uses power up behaviour of SRAM cells & each cell consists of 6 transistors

→ 2 cross coupled inverters produce stable state ($\begin{matrix} A=1 & B=0 \\ \text{or} & \\ A=0 & B=1 \end{matrix}$)

→ During startup, both nodes try to go high but due to transistor mismatch, only one node wins

This startup pattern becomes PUF fingerprint



→ These variations in transistors come from process variation

→ It has very small CRP space ⇒ weak PUF

→ Advantages :

→ Already available in many chips

→ No extra hardware

→ Low Power

→ Disadvantages :

→ Some SRAM cells are unstable & can flip bits

→ Butterfly PUF

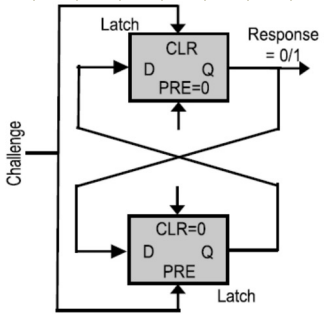
→ Designed mainly for FPGAs

→ It uses metastability of cross-coupled latches

→ Architecture

2 Cross-Coupled Latches
↓
Excited by Challenge Signal
↓
Metastable state resolves randomly
↓
Output bit

→ Similar to SRAM PUF but latch instead of memory cells



→ Applications

1) Secret Key Generation
2) Device Auth.
3) Software & IP Licensing
4) Hardware metering

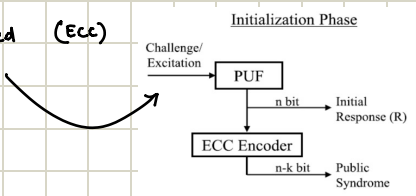
→ Secret Key Generation

→ PUFs generate encryption keys & session keys but responses must be error free

→ Requires $HD_{intra} = 0$ (High Reproducability)

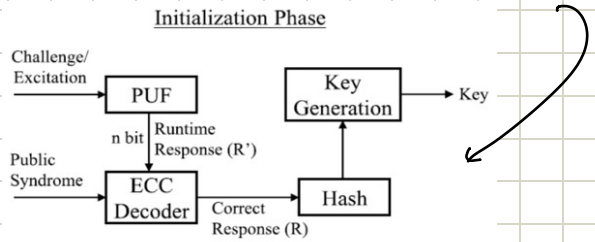
→ In case environmental noise flips the bits, Error Correcting Codes are used (ECC)

ex: PUF Response → ECC correction → Stable Key



→ PUF o/p may not satisfy requirements, so PUF o/p is passed through Hash function to produce secure Key

ex: SHA



→ The key advantage is PUF Keys are never stored, instead generated from hardware making it extremely difficult for extraction

True Random Number Generator (TRNG)

→ Its a hardware system that produces unpredictable random bits which are generated from physical randomness. They are completely undeterministic

→ TRNG are used in cryptographic key generation, session keys, one-time pads, nonces etc.,

→ **Characteristics**

i) Unpredictable - Next bit can't be predicted from prev. bit

ii) Uniform distribution - Prob (0) = Prob (1)

iii) Independance - Each bit is independant of prev. bit

→ **Source of Randomness (Entropy)**

→ TRNG relies on physical noise sources in hardware .

ex: Random Telegraph Noise , Power supply noise , Thermal noise , Radioactive Decay etc.,

→ **TRNG Internal Processing Stages**

3 stages :

1. Entropy Source

The entropy source generates **physical randomness.**

Examples:

- transistor noise
- oscillator jitter
- metastability
- thermal noise

This randomness is **analog in nature.**

2. Sampling / Extraction Unit

The sampling unit converts the **analog randomness into digital bits.**

Main characteristics:

- extracts entropy from the noise source
- converts it to digital `0` or `1`
- must **not disturb the original noise process**

Design goal:

```
code

maximize entropy
while minimizing area and power
```

However, this stage usually **does not guarantee perfect randomness.** It simply captures raw random bits.

3. Post-Processing Unit

The raw output from the sampling unit often contains:

- bias
- correlations
- uneven distribution of 0 and 1

The **post-processing stage improves randomness.**

Typical techniques:

Technique	Purpose
Von Neumann extractor	removes bias
Cryptographic hash	randomizes output
Stream cipher	improves statistical randomness

Modern TRNGs may also include:

- noise conditioning circuits
- side-channel attack protection
- fault-tolerance blocks.

→ **TRNG vs PRNG (Pseudo RNG)**

Feature	TRNG	PRNG
Source	physical noise	algorithm
Predictability	unpredictable	predictable
Security	high	lower
Speed	slower	faster

Relies on initial seed

if attacker knows than, then they can gain access

→ **TRNG Quality Properties**

3. TRNG Quality Properties

Unlike PUFs, TRNG evaluation mainly focuses on **randomness.**

Randomness Testing

Two commonly used statistical test suites:

- **NIST Test Suite**
- **DieHARD Test Suite**

These detect:

- bias
- correlations
- patterns
- predictability.

Even good entropy sources may produce biased outputs under:

- temperature variation
- voltage fluctuation
- process variation.

Randomness Improvement

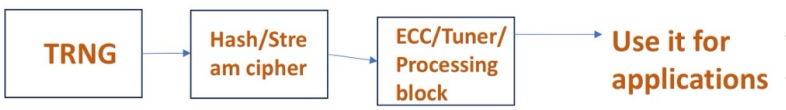
To correct bias and improve entropy:

- cryptographic hash functions
- Von Neumann corrector
- stream ciphers

These manipulate the raw output to produce:

```
code                                    Copy

uniform
statistically random
bitstreams
```



→ Common TRNG (CMOS based TRNGs)

1) Noise-based TRNG
2) Oscillator-based TRNG
3) Memory-based TRNG
4) Chaos TRNG
5) Quantum TRNG

→ Noise based TRNG

Principle

Uses **device intrinsic electrical noise**.

Common sources:

- Random Telegraph Noise (RTN)
- Zener diode noise
- Flicker noise (1/f noise)
- Johnson thermal noise.

Working

1. Noise produces random analog voltage.
2. Voltage is sampled periodically.
3. Compared to a threshold.

```
code

Voltage > Threshold → 1
Voltage < Threshold → 0
```

The threshold is adjusted to obtain:

```
code

P(0) ≈ P(1)
```

Example Implementation

Intel TRNG used **Johnson thermal noise.**

A simpler implementation uses:

```
code

cross-coupled inverter pair
```

Both nodes are driven simultaneously.

Random noise decides which node becomes `1` or `0`.

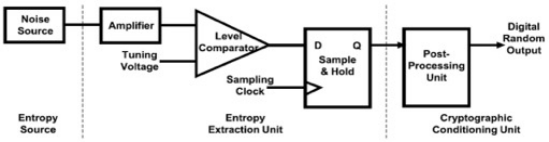
Limitations

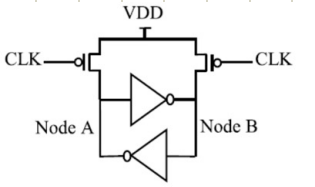
Noise-based TRNGs face several issues:

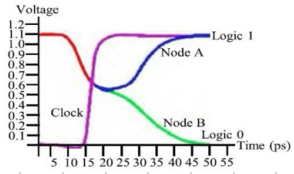
- noise is difficult to measure
- analog amplification required
- process improvements reduce noise sources
- amplifier imperfections affect randomness.

Additionally:

- electromagnetic interference may synchronize nearby RNGs
- entropy may reduce.







→ Oscillator - based TRNG

Principle

Uses **jitter in ring oscillators**.

A ring oscillator consists of an **odd number of inverters connected in a loop.**

```
code

NOT → NOT → NOT → feedback
```

It oscillates continuously.

Source of Randomness

Noise in the circuit causes **jitter in oscillation timing**.

Therefore the exact time of signal transitions becomes unpredictable.

Operation

1. Several ring oscillators run simultaneously.
2. Their outputs are sampled.
3. XOR operations combine outputs to increase entropy.

```
code

RO1 ⊕ RO2 ⊕ RO3 → Random bit
```

Noise Augmentation

To increase entropy:

```
code

Noise-Augmenting Ring Oscillators (NAROs)
```

are added.

These create additional power supply noise.

Activation may be controlled using an **LFSR**.

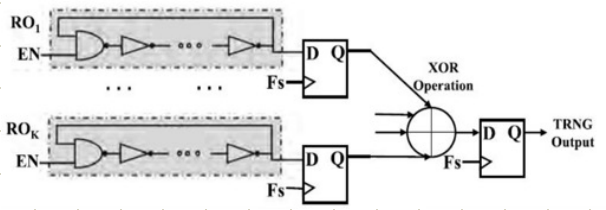
Limitations

- jitter may be small in modern processes
- randomness difficult to prove
- environmental effects influence oscillation frequency.

Possible solutions include:

- Fibonacci ring oscillators
- Galois ring oscillators.

These provide variable oscillator lengths.



→ Memory Based TRNG

Uses **SRAM power-up randomness.**

When SRAM powers up:

```
code

cells randomly initialize to 0 or 1
```

due to:

- process variation
- noise
- environmental conditions.

Difference from SRAM-PUF

SRAM-PUF	SRAM-TRNG
uses stable startup states	uses unstable startup states
generates fingerprint	generates random numbers

Cells that show **equal probability of 0 and 1** are used.

Limitations

- randomness depends on technology
- affected by environmental conditions
- power-cycling required for new random numbers.

→ TRNG Applications

Kerckhoffs' Principle

In cryptographic systems, security should depend **only on the secrecy of the key**, not on the secrecy of the algorithm.

```
text
Security = secrecy of key
NOT secrecy of algorithm
```

Therefore, the **quality of the cryptographic key** is critical.

Most cryptographic keys are generated from **random numbers**. Hence, the randomness provided by a TRNG directly affects the **strength of the entire security system**.

Importance of True Random Keys

If a key is **perfectly random**:

- the only possible attack is **brute-force search**
- attacker must try every possible key

Example:

For a 128-bit key:

```
code
Possible keys = 2^128
```

This makes brute-force attacks computationally infeasible.

However, if the key is **predictable or biased**, the attacker's search space becomes smaller, weakening the system.

→ Cryptographic Key Generation
→ Initialization Vectors
→ Session Key Generation
→ Digital Signature Systems
→ Authentication Protocols
→ Msg Authentication Codes
→ Nonce Generation
→ One-time Pad Encryption
→ Pin & Password Generation
→ PUF Challenge Generation

→ Existing Challenges & Attacks

1) Modelling Attacks against strong PUFs
2) Environmental & Aging impacts on PUFs
3) Cloning SRAM PUF

5. Modeling Attacks on Strong PUFs

Strong PUFs generate a large number of **Challenge-Response Pairs (CRPs)**.

```
code
Challenge → PUF → Response
```

However, attackers may exploit this property.

Attack Procedure

1. Attacker collects a **subset of CRPs**.
2. Uses machine learning to build a **predictive model**.
3. Predicts responses for unseen challenges.

This allows the attacker to impersonate the device.

Example: Arbiter PUF Modeling Attack

Arbiter PUF delay behavior can be approximated using a **linear delay model**.

Total delay:

```
code
Total delay = sum of stage delays
```

The final response depends on the **delay difference between two paths**.

Machine learning algorithms can learn this relationship.

Solution: XOR Arbiter PUF

To prevent modeling attacks, designers introduce **nonlinearity**.

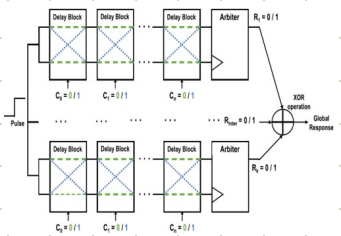
One approach:

```
code
Multiple Arbiter PUFs
        ↓
Outputs XORed
        ↓
Final Response
```

This is called an **XOR Arbiter PUF**.

Advantages:

- increased complexity
- harder for machine learning models to predict.



6. Environmental and Aging Effects on PUF

PUF outputs may change due to:

- temperature variation
- power supply noise
- transistor aging
- wear-out mechanisms.

These effects reduce **PUF reliability**.

Example: RO-PUF Frequency Degradation

In Ring Oscillator PUF:

```
code
ROx frequency = fxi
ROy frequency = fyi
```

Response rule:

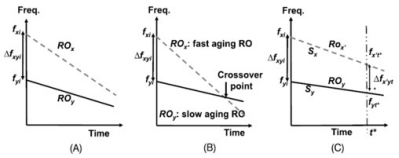
```
code
fxi > fyi → output = 1
fxi < fyi → output = 0
```

However, aging can change frequencies.

Example:

```
code
Initially: fxi > fyi → output = 1
After aging: fxi < fyi → output flips
```

This produces **bit-flip errors**.

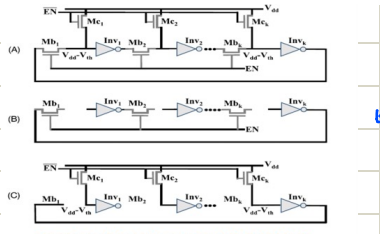


Bit-Flip due to Frequency degradation in RO pair

(A) ROs with moderate degradation (stable pair)

(B) ROs with high degradation (unstable pair)

(C) Ros with negligible degradation (highly stable pair)



Operation of the aging-resistant RO (ARO) PUF

(A) Schematic

(B) Activity in oscillatory mode

(C) Activity in non-oscillatory mode

7. Error Correction in PUF

To maintain reliable output, **Error Correcting Codes (ECC)** are used.

ECC helps correct noise-induced bit errors.

Process:

```
code
PUF response
     ↓
ECC correction
     ↓
Stable response
```

However ECC introduces:

- extra hardware
- power overhead
- area overhead.

8. Aging-Resistant RO-PUF

Researchers proposed **Aging Resistant RO-PUF (ARO-PUF)**.

It reduces transistor aging effects.

Two operating modes:

Oscillatory Mode

```
code
EN = 1
```

Normal PUF operation.

Non-Oscillatory Mode

```
code
EN = 0
```

PUF is idle.

Stress is reduced by:

- removing DC stress (prevents **NBTI**)
- removing AC switching stress (prevents **HCI**).

This improves long-term reliability.

9. Environmental Effects on TRNG

TRNG randomness can be affected by environmental conditions.

Examples:

Voltage Manipulation Attack

Attacker changes supply voltage:

```
code
Vdd variation → biased random bits
```

Temperature Attack

Thermal noise depends on temperature.

- Low temperature → less noise → less randomness
- High temperature → more randomness.

Frequency Injection Attack

External signals interfere with oscillator jitter.

This reduces entropy.

Electromagnetic Attack

External EM signals leak information about random number generation.